

Untitled

2026-08-02

```
# =====  
# R Code for Power Analysis and Simulation Studies  
# A Semiparametric Alternative to the Log-Rank Test with Right-Censored Data  
# =====  
  
# Load required packages  
library(survival)  
library(emplik)  
  
## Warning: package 'emplik' was built under R version 4.6.1  
  
library(parallel)  
library(doParallel)  
  
## Warning: package 'doParallel' was built under R version 4.6.1  
  
## Loading required package: foreach  
  
## Loading required package: iterators  
  
library(ggplot2)  
library(tidyr)  
library(dplyr)  
  
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union  
  
# =====  
# Section 1: Helper Functions for DRM-Based Test  
# =====  
  
#' Compute profile empirical likelihood for DRM with right-censored data  
#'
```

```

#' @param beta Parameter vector
#' @param hmat Basis function matrix (n x p)
#' @param group Group indicator (0/1)
#' @param delta Censoring indicator (1=event, 0=censored)
#' @param time Observed time
#' @param rho n2/n1 ratio
#' @param tol Convergence tolerance
#' @param maxiter Maximum iterations
#' @return Profile log-likelihood value
profile_el_censored <- function(beta, hmat, group, delta, time, rho,
                                tol = 1e-6, maxiter = 100) {

  n <- length(group)
  n1 <- sum(group == 0)
  n2 <- sum(group == 1)

  # Get unique observed times
  unique_times <- sort(unique(time[delta == 1]))
  k <- length(unique_times)

  if (k == 0) return(-Inf)

  # Count events and censored observations at each time
  event_counts <- matrix(0, nrow = k, ncol = 2)
  cens_counts <- matrix(0, nrow = k, ncol = 2)

  for (j in 1:k) {
    t_j <- unique_times[j]
    event_counts[j, 1] <- sum(group == 0 & time == t_j & delta == 1)
    event_counts[j, 2] <- sum(group == 1 & time == t_j & delta == 1)
    cens_counts[j, 1] <- sum(group == 0 & time == t_j & delta == 0)
    cens_counts[j, 2] <- sum(group == 1 & time == t_j & delta == 0)
  }

  # Compute h values at unique times
  h_unique <- matrix(NA, nrow = k, ncol = ncol(hmat))
  for (j in 1:k) {
    idx <- which(time == unique_times[j] & delta == 1)[1]
    if (!is.na(idx)) {
      h_unique[j, ] <- hmat[idx, ]
    }
  }

  # Initialize p
  p <- rep(1/k, k)

  # Iterative optimization for p given beta
  for (iter in 1:maxiter) {
    p_old <- p

    # Compute exponential terms
    exp_term <- exp(beta %*% t(h_unique))
    if (ncol(h_unique) == 1) {
      exp_term <- as.vector(exp_term)
    }
  }
}

```

```

} else {
  exp_term <- as.vector(exp_term)
}

# Compute survival probabilities
S1 <- cumsum(rev(p))[k:1]
S2 <- cumsum(rev(p * exp_term))[k:1]

# Update p using fixed point iteration
numerator <- event_counts[,1] + event_counts[,2]
denom_events <- (event_counts[,1] + event_counts[,2] * exp_term) / p

# Add censoring contribution
p <- numerator / denom_events

# Normalize
p <- p / sum(p)
p <- pmax(p, 1e-10)

if (max(abs(p - p_old)) < tol) break
}

# Compute profile log-likelihood
exp_term <- as.vector(exp(beta %*% t(h_unique)))
S1 <- cumsum(rev(p))[k:1]
S2 <- cumsum(rev(p * exp_term))[k:1]

loglik <- 0
for (j in 1:k) {
  if (event_counts[j,1] > 0) loglik <- loglik + event_counts[j,1] * log(p[j])
  if (event_counts[j,2] > 0) loglik <- loglik + event_counts[j,2] * log(exp_term[j] * p[j])
  if (cens_counts[j,1] > 0) loglik <- loglik + cens_counts[j,1] * log(S1[j])
  if (cens_counts[j,2] > 0) loglik <- loglik + cens_counts[j,2] * log(S2[j])
}

return(loglik)
}

#' DRM-based likelihood ratio test for two-sample comparison with right-censored data
#'
#' @param time Observed time vector
#' @param delta Censoring indicator (1=event, 0=censored)
#' @param group Group indicator (0/1)
#' @param h_fun Basis function(s) to use (can be function or list of functions)
#' @param basis_type Type of basis: "log", "linear", "both", "boxcox"
#' @return List with test statistic, p-value, and beta estimate
drm_lrt_test <- function(time, delta, group, h_fun = NULL,
                        basis_type = "log", boxcox_r = NULL) {

  n <- length(time)
  n1 <- sum(group == 0)
  n2 <- sum(group == 1)
  rho <- n2 / n1

```

```

# Construct basis functions
if (is.null(h_fun)) {
  if (basis_type == "log") {
    hmat <- cbind(log(pmax(time, 1e-10)))
  } else if (basis_type == "linear") {
    hmat <- cbind(time)
  } else if (basis_type == "both") {
    hmat <- cbind(log(pmax(time, 1e-10)), time)
  } else if (basis_type == "boxcox") {
    if (is.null(boxcox_r)) {
      # Estimate r using profile likelihood
      r_values <- seq(-1, 2, by = 0.1)
      ll_values <- numeric(length(r_values))
      for (i in seq_along(r_values)) {
        r <- r_values[i]
        if (abs(r) < 1e-6) {
          h_temp <- cbind(log(pmax(time, 1e-10)))
        } else {
          h_temp <- cbind((time^r - 1) / r)
        }
        # Optimize beta
        opt <- try(optim(par = 0, fn = function(b) {
          -profile_el_censored(b, h_temp, group, delta, time, rho)
        }, method = "BFGS"), silent = TRUE)
        if (!inherits(opt, "try-error") && opt$convergence == 0) {
          ll_values[i] <- -opt$value
        } else {
          ll_values[i] <- -Inf
        }
      }
      best_r <- r_values[which.max(ll_values)]
      if (abs(best_r) < 1e-6) {
        hmat <- cbind(log(pmax(time, 1e-10)))
      } else {
        hmat <- cbind((time^best_r - 1) / best_r)
      }
      boxcox_r <- best_r
    } else {
      if (abs(boxcox_r) < 1e-6) {
        hmat <- cbind(log(pmax(time, 1e-10)))
      } else {
        hmat <- cbind((time^boxcox_r - 1) / boxcox_r)
      }
    }
  } else {
    stop("Unknown basis_type")
  }
} else {
  if (is.function(h_fun)) {
    hmat <- cbind(h_fun(time))
  } else {
    hmat <- do.call(cbind, lapply(h_fun, function(f) f(time)))
  }
}

```

```

}

p <- ncol(hmat)

# Compute null log-likelihood (beta = 0)
ll_null <- profile_el_censored(rep(0, p), hmat, group, delta, time, rho)

# Optimize beta
opt <- try(optim(par = rep(0, p), fn = function(b) {
  -profile_el_censored(b, hmat, group, delta, time, rho)
}, method = "BFGS", control = list(maxit = 500)), silent = TRUE)

if (inherits(opt, "try-error") || opt$convergence != 0) {
  # Try with different starting values
  opt <- try(optim(par = rep(0.1, p), fn = function(b) {
    -profile_el_censored(b, hmat, group, delta, time, rho)
  }, method = "BFGS", control = list(maxit = 500)), silent = TRUE)
}

if (inherits(opt, "try-error") || opt$convergence != 0) {
  return(list(statistic = NA, p_value = NA, beta_hat = NA, converged = FALSE))
}

beta_hat <- opt$par
ll_alt <- -opt$value

# Likelihood ratio test statistic
lr_stat <- -2 * (ll_null - ll_alt)
lr_stat <- max(0, lr_stat) # Ensure non-negative

p_value <- pchisq(lr_stat, df = p, lower.tail = FALSE)

return(list(
  statistic = lr_stat,
  p_value = p_value,
  beta_hat = beta_hat,
  df = p,
  converged = TRUE,
  boxcox_r = ifelse(exists("boxcox_r"), boxcox_r, NA)
))
}

#' Wrapper for standard log-rank test
#'
#' @param time Observed time vector
#' @param delta Censoring indicator
#' @param group Group indicator
#' @return List with statistic and p-value
log_rank_test <- function(time, delta, group) {
  fit <- survdiff(Surv(time, delta) ~ group)
  return(list(
    statistic = fit$chisq,
    p_value = 1 - pchisq(fit$chisq, df = 1)
  ))
}

```

```

))
}

#' Wrapper for weighted log-rank tests (Peto-Prentice/Gehan)
#'
#' @param time Observed time vector
#' @param delta Censoring indicator
#' @param group Group indicator
#' @param weight_type Type of weighting: "peto", "gehan", "fh"
#' @param rho Parameter for Fleming-Harrington
#' @param gamma Parameter for Fleming-Harrington
#' @return List with statistic and p-value
weighted_log_rank_test <- function(time, delta, group,
                                   weight_type = "gehan",
                                   rho = 0, gamma = 1) {
  # Using survMisc for weighted tests
  if (!requireNamespace("survMisc", quietly = TRUE)) {
    warning("survMisc package not available. Using fallback.")
    # Simple fallback using coxph
    fit <- coxph(Surv(time, delta) ~ group)
    return(list(
      statistic = summary(fit)$score,
      p_value = summary(fit)$sctest[3]
    ))
  }

  # Compute weighted log-rank
  fit <- survdiff(Surv(time, delta) ~ group, rho = ifelse(weight_type == "peto", 1, 0))

  # For Gehan test, use different weighting
  if (weight_type == "gehan") {
    # Gehan test is equivalent to Wilcoxon
    fit <- survdiff(Surv(time, delta) ~ group, rho = 1)
  }

  if (weight_type == "fh") {
    # Fleming-Harrington test requires different approach
    # Use coxph with time-dependent weights
    fit <- coxph(Surv(time, delta) ~ group)
    # Approximate with score test
  }

  return(list(
    statistic = fit$chisq,
    p_value = 1 - pchisq(fit$chisq, df = 1)
  ))
}

```

```

# =====
# Section 2: Data Generation Functions for Each Scenario
# =====

#' Generate data for Scenario A: Proportional Hazards

```

```

#'
#' @param n Sample size per group
#' @param delta_hazard Hazard ratio (lambda2/lambda1)
#' @param cens_prop Censoring proportion
#' @return List with time, delta, group
generate_scenario_A <- function(n, delta_hazard, cens_prop = 0.2) {
  # Group 1: Exponential(lambda=1)
  t1 <- rexp(n, rate = 1)

  # Group 2: Exponential(lambda=delta_hazard)
  t2 <- rexp(n, rate = delta_hazard)

  # Generate independent censoring
  # Find appropriate censoring rate to achieve desired censoring proportion
  if (cens_prop > 0) {
    # For exponential, we can derive censoring rate
    #  $P(\text{censoring}) = \lambda_c / (\lambda + \lambda_c)$ 
    # Solve for lambda_c
    lambda_c1 <- cens_prop * 1 / (1 - cens_prop)
    lambda_c2 <- cens_prop * delta_hazard / (1 - cens_prop)
    # Use same censoring distribution for both groups
    lambda_c <- mean(c(lambda_c1, lambda_c2))
    c1 <- rexp(n, rate = lambda_c)
    c2 <- rexp(n, rate = lambda_c)
  } else {
    c1 <- rep(Inf, n)
    c2 <- rep(Inf, n)
  }

  time1 <- pmin(t1, c1)
  delta1 <- as.numeric(t1 <= c1)

  time2 <- pmin(t2, c2)
  delta2 <- as.numeric(t2 <= c2)

  return(list(
    time = c(time1, time2),
    delta = c(delta1, delta2),
    group = c(rep(0, n), rep(1, n))
  ))
}

#' Generate data for Scenario B: Crossing Hazards
#'
#' @param n Sample size per group
#' @param cens_prop Censoring proportion
#' @param gamma Scale parameter for group 2 Weibull
#' @return List with time, delta, group
generate_scenario_B <- function(n, cens_prop = 0.2, gamma = 0.5) {
  # Group 1: Weibull(shape=2, scale=1)
  t1 <- rweibull(n, shape = 2, scale = 1)

  # Group 2: Weibull(shape=0.5, scale=gamma)

```

```

t2 <- rweibull(n, shape = 0.5, scale = gamma)

# Generate censoring
if (cens_prop > 0) {
  # Use uniform censoring to achieve desired proportion
  max_t <- max(c(t1, t2)) * 1.5
  c1 <- runif(n, 0, max_t)
  c2 <- runif(n, 0, max_t)
} else {
  c1 <- rep(Inf, n)
  c2 <- rep(Inf, n)
}

time1 <- pmin(t1, c1)
delta1 <- as.numeric(t1 <= c1)

time2 <- pmin(t2, c2)
delta2 <- as.numeric(t2 <= c2)

# Adjust censoring proportion if needed
# We'll just use the generated censoring

return(list(
  time = c(time1, time2),
  delta = c(delta1, delta2),
  group = c(rep(0, n), rep(1, n))
))
}

#' Generate data for Scenario C: Long-term Survivors
#'
#' @param n Sample size per group
#' @param delta_hazard Hazard ratio for non-cured component
#' @param cure_prop Cured proportion
#' @param cens_prop Censoring proportion
#' @return List with time, delta, group
generate_scenario_C <- function(n, delta_hazard, cure_prop = 0.2, cens_prop = 0.2) {
  # Group 1: 80% Exponential(lambda=1), 20% cured
  u1 <- runif(n)
  t1 <- ifelse(u1 < cure_prop, Inf, rexp(n, rate = 1))

  # Group 2: 80% Exponential(lambda=delta_hazard), 20% cured
  u2 <- runif(n)
  t2 <- ifelse(u2 < cure_prop, Inf, rexp(n, rate = delta_hazard))

  # Generate censoring
  if (cens_prop > 0) {
    # Use uniform censoring
    max_t <- quantile(c(t1[t1 < Inf], t2[t2 < Inf]), 0.95)
    c1 <- runif(n, 0, max_t * 1.2)
    c2 <- runif(n, 0, max_t * 1.2)
  } else {
    c1 <- rep(Inf, n)

```



```

    c2 <- rep(Inf, n)
  }

  time1 <- pmin(t1, c1)
  delta1 <- as.numeric(t1 <= c1 & t1 < Inf)

  time2 <- pmin(t2, c2)
  delta2 <- as.numeric(t2 <= c2 & t2 < Inf)

  # Replace Inf with large value for censored cured patients
  time1[is.infinite(time1)] <- c1[is.infinite(time1)]
  time2[is.infinite(time2)] <- c2[is.infinite(time2)]

  return(list(
    time = c(time1, time2),
    delta = c(delta1, delta2),
    group = c(rep(0, n), rep(1, n))
  ))
}

#' Generate data for Scenario D: Log-normal Distributions
#'
#' @param n Sample size per group
#' @param delta_mean Difference in log-mean
#' @param sigma Common log-normal scale
#' @param cens_prop Censoring proportion
#' @return List with time, delta, group
generate_scenario_D <- function(n, delta_mean, sigma = 1, cens_prop = 0.2) {
  # Group 1: Log-normal(mu=0, sigma=1)
  t1 <- rlnorm(n, meanlog = 0, sdlog = sigma)

  # Group 2: Log-normal(mu=delta_mean, sigma=1)
  t2 <- rlnorm(n, meanlog = delta_mean, sdlog = sigma)

  # Generate censoring
  if (cens_prop > 0) {
    max_t <- quantile(c(t1, t2), 0.9) * 1.5
    c1 <- runif(n, 0, max_t)
    c2 <- runif(n, 0, max_t)
  } else {
    c1 <- rep(Inf, n)
    c2 <- rep(Inf, n)
  }

  time1 <- pmin(t1, c1)
  delta1 <- as.numeric(t1 <= c1)

  time2 <- pmin(t2, c2)
  delta2 <- as.numeric(t2 <= c2)

  return(list(
    time = c(time1, time2),
    delta = c(delta1, delta2),

```

```

    group = c(rep(0, n), rep(1, n))
  })
}

#' Generate data for Scenario E: Log-normal with Different Variances
#'
#' @param n Sample size per group
#' @param sigma1 Variance for group 1
#' @param sigma2 Variance for group 2
#' @param cens_prop Censoring proportion
#' @return List with time, delta, group
generate_scenario_E <- function(n, sigma1 = 1, sigma2 = 1.5, cens_prop = 0.2) {
  # Group 1: Log-normal(mu=0, sigma=sigma1)
  t1 <- rlnorm(n, meanlog = 0, sdlog = sigma1)

  # Group 2: Log-normal(mu=0.5, sigma=sigma2)
  t2 <- rlnorm(n, meanlog = 0.5, sdlog = sigma2)

  # Generate censoring
  if (cens_prop > 0) {
    max_t <- quantile(c(t1, t2), 0.9) * 1.5
    c1 <- runif(n, 0, max_t)
    c2 <- runif(n, 0, max_t)
  } else {
    c1 <- rep(Inf, n)
    c2 <- rep(Inf, n)
  }

  time1 <- pmin(t1, c1)
  delta1 <- as.numeric(t1 <= c1)

  time2 <- pmin(t2, c2)
  delta2 <- as.numeric(t2 <= c2)

  return(list(
    time = c(time1, time2),
    delta = c(delta1, delta2),
    group = c(rep(0, n), rep(1, n))
  ))
}

```

```

# =====
# Section 3: Simulation Functions
# =====

#' Run single simulation replication
#'
#' @param data_gen_func Function to generate data
#' @param data_gen_args Arguments for data generation
#' @param test_methods List of test methods to apply
#' @return Data frame with results
run_simulation_replication <- function(data_gen_func, data_gen_args,
                                     test_methods = c("drm_log", "drm_both", "logrank")) {

```

```

# Generate data
data <- do.call(data_gen_func, data_gen_args)

results <- data.frame()

# DRM with log basis
if ("drm_log" %in% test_methods) {
  tryCatch({
    drm_res <- drm_lrt_test(data$time, data$delta, data$group, basis_type = "log")
    results <- rbind(results, data.frame(
      method = "DRM-log",
      statistic = drm_res$statistic,
      p_value = drm_res$p_value,
      reject = drm_res$p_value < 0.05,
      converged = drm_res$converged
    ))
  }, error = function(e) {
    results <- rbind(results, data.frame(
      method = "DRM-log",
      statistic = NA,
      p_value = NA,
      reject = NA,
      converged = FALSE
    ))
  })
}

# DRM with both log and linear basis
if ("drm_both" %in% test_methods) {
  tryCatch({
    drm_res <- drm_lrt_test(data$time, data$delta, data$group, basis_type = "both")
    results <- rbind(results, data.frame(
      method = "DRM-both",
      statistic = drm_res$statistic,
      p_value = drm_res$p_value,
      reject = drm_res$p_value < 0.05,
      converged = drm_res$converged
    ))
  }, error = function(e) {
    results <- rbind(results, data.frame(
      method = "DRM-both",
      statistic = NA,
      p_value = NA,
      reject = NA,
      converged = FALSE
    ))
  })
}

# DRM with Box-Cox transformation
if ("drm_boxcox" %in% test_methods) {
  tryCatch({
    drm_res <- drm_lrt_test(data$time, data$delta, data$group, basis_type = "boxcox")

```

```

    results <- rbind(results, data.frame(
      method = "DRM-boxcox",
      statistic = drm_res$statistic,
      p_value = drm_res$p_value,
      reject = drm_res$p_value < 0.05,
      converged = drm_res$converged
    ))
  }, error = function(e) {
    results <- rbind(results, data.frame(
      method = "DRM-boxcox",
      statistic = NA,
      p_value = NA,
      reject = NA,
      converged = FALSE
    ))
  })
}

# Standard log-rank test
if ("logrank" %in% test_methods) {
  tryCatch({
    lr_res <- log_rank_test(data$time, data$delta, data$group)
    results <- rbind(results, data.frame(
      method = "Log-rank",
      statistic = lr_res$statistic,
      p_value = lr_res$p_value,
      reject = lr_res$p_value < 0.05,
      converged = TRUE
    ))
  }, error = function(e) {
    results <- rbind(results, data.frame(
      method = "Log-rank",
      statistic = NA,
      p_value = NA,
      reject = NA,
      converged = FALSE
    ))
  })
}

# Peto-Prentice test
if ("peto" %in% test_methods) {
  tryCatch({
    peto_res <- weighted_log_rank_test(data$time, data$delta, data$group,
                                       weight_type = "peto")
    results <- rbind(results, data.frame(
      method = "Peto-Prentice",
      statistic = peto_res$statistic,
      p_value = peto_res$p_value,
      reject = peto_res$p_value < 0.05,
      converged = TRUE
    ))
  }, error = function(e) {

```

```

    results <- rbind(results, data.frame(
      method = "Peto-Prentice",
      statistic = NA,
      p_value = NA,
      reject = NA,
      converged = FALSE
    ))
  })
}

# Gehan test
if ("gehan" %in% test_methods) {
  tryCatch({
    gehan_res <- weighted_log_rank_test(data$time, data$delta, data$group,
                                         weight_type = "gehan")

    results <- rbind(results, data.frame(
      method = "Gehan",
      statistic = gehan_res$statistic,
      p_value = gehan_res$p_value,
      reject = gehan_res$p_value < 0.05,
      converged = TRUE
    ))
  }, error = function(e) {
    results <- rbind(results, data.frame(
      method = "Gehan",
      statistic = NA,
      p_value = NA,
      reject = NA,
      converged = FALSE
    ))
  })
}

return(results)
}

#' Run simulation for a specific scenario
#'
#' @param scenario_name Name of scenario
#' @param n Sample size per group
#' @param n_replications Number of replications
#' @param data_gen_func Data generation function
#' @param data_gen_args List of arguments for data generation
#' @param test_methods Vector of test methods to include
#' @param n_cores Number of cores for parallel processing
#' @return Data frame with simulation results
run_simulation <- function(scenario_name, n, n_replications = 1000,
                          data_gen_func, data_gen_args,
                          test_methods = c("drm_log", "drm_both", "drm_boxcox",
                                           "logrank", "peto", "gehan"),
                          n_cores = 4) {

  cat(sprintf("\nRunning simulation for %s (n=%d, reps=%d)...\n",

```

```

        scenario_name, n, n_replications))

# Setup parallel processing
cl <- makeCluster(min(n_cores, detectCores() - 1))
registerDoParallel(cl)

# Run replications in parallel
results <- foreach(rep = 1:n_replications, .combine = rbind,
                   .packages = c("survival", "emplik")) %dopar% {

  # Run single replication
  tryCatch({
    run_simulation_replication(data_gen_func, data_gen_args, test_methods)
  }, error = function(e) {
    # Return empty data frame on error
    data.frame()
  })
}

stopCluster(cl)

# Add scenario and sample size information
if (nrow(results) > 0) {
  results$scenario <- scenario_name
  results$n <- n
}

return(results)
}

#' Calculate power from simulation results
#'
#' @param results Data frame from simulation
#' @param alpha Significance level
#' @return Summary data frame with power and convergence rates
calculate_power <- function(results, alpha = 0.05) {
  summary <- results %>%
    group_by(scenario, n, method) %>%
    summarise(
      power = mean(reject, na.rm = TRUE),
      convergence_rate = mean(converged, na.rm = TRUE),
      mean_statistic = mean(statistic, na.rm = TRUE),
      n_valid = sum(!is.na(reject)),
      .groups = "drop"
    )
  return(summary)
}

# =====
# Section 4: Run All Five Scenarios
# =====

#' Run complete simulation study for all scenarios

```

```

#'
#' @param n Sample size per group
#' @param n_replications Number of replications
#' @param cens_prop Censoring proportion
#' @param n_cores Number of cores
#' @return Combined results for all scenarios
run_full_simulation <- function(n = 200, n_replications = 1000,
                               cens_prop = 0.2, n_cores = 4) {

  all_results <- data.frame()

  # Scenario A: Proportional Hazards
  for (delta in c(1.2, 1.5, 2.0)) {
    cat(sprintf("\nScenario A: PH with delta=%.1f\n", delta))
    results <- run_simulation(
      scenario_name = sprintf("A_PH_delta=%.1f", delta),
      n = n,
      n_replications = n_replications,
      data_gen_func = generate_scenario_A,
      data_gen_args = list(n = n, delta_hazard = delta, cens_prop = cens_prop),
      test_methods = c("drm_log", "drm_both", "drm_boxcox", "logrank", "peto", "gehan"),
      n_cores = n_cores
    )
    all_results <- rbind(all_results, results)
  }

  # Scenario B: Crossing Hazards
  for (gamma in c(0.5, 0.7, 1.0)) {
    cat(sprintf("\nScenario B: Crossing Hazards with gamma=%.1f\n", gamma))
    results <- run_simulation(
      scenario_name = sprintf("B_Crossing_gamma=%.1f", gamma),
      n = n,
      n_replications = n_replications,
      data_gen_func = generate_scenario_B,
      data_gen_args = list(n = n, cens_prop = cens_prop, gamma = gamma),
      test_methods = c("drm_log", "drm_both", "drm_boxcox", "logrank", "peto", "gehan"),
      n_cores = n_cores
    )
    all_results <- rbind(all_results, results)
  }

  # Scenario C: Long-term Survivors
  for (delta in c(1.5, 2.0)) {
    for (cure in c(0.15, 0.25)) {
      cat(sprintf("\nScenario C: Long-term with delta=%.1f, cure=%.2f\n", delta, cure))
      results <- run_simulation(
        scenario_name = sprintf("C_Longterm_delta=%.1f_cure=%.2f", delta, cure),
        n = n,
        n_replications = n_replications,
        data_gen_func = generate_scenario_C,
        data_gen_args = list(n = n, delta_hazard = delta,
                              cure_prop = cure, cens_prop = cens_prop),
        test_methods = c("drm_log", "drm_both", "drm_boxcox", "logrank", "peto", "gehan"),

```

```

    n_cores = n_cores
  )
  all_results <- rbind(all_results, results)
}
}

# Scenario D: Log-normal Distributions
for (delta_mean in c(0.3, 0.5, 0.7)) {
  cat(sprintf("\nScenario D: Log-normal with delta_mean=%.1f\n", delta_mean))
  results <- run_simulation(
    scenario_name = sprintf("D_Lognormal_delta=%.1f", delta_mean),
    n = n,
    n_replications = n_replications,
    data_gen_func = generate_scenario_D,
    data_gen_args = list(n = n, delta_mean = delta_mean,
                        sigma = 1, cens_prop = cens_prop),
    test_methods = c("drm_log", "drm_both", "drm_boxcox", "logrank", "peto", "gehan"),
    n_cores = n_cores
  )
  all_results <- rbind(all_results, results)
}

# Scenario E: Different Variances
for (sigma2 in c(1.2, 1.5, 2.0)) {
  cat(sprintf("\nScenario E: Different Variances with sigma2=%.1f\n", sigma2))
  results <- run_simulation(
    scenario_name = sprintf("E_Variance_sigma2=%.1f", sigma2),
    n = n,
    n_replications = n_replications,
    data_gen_func = generate_scenario_E,
    data_gen_args = list(n = n, sigma1 = 1, sigma2 = sigma2,
                        cens_prop = cens_prop),
    test_methods = c("drm_log", "drm_both", "drm_boxcox", "logrank", "peto", "gehan"),
    n_cores = n_cores
  )
  all_results <- rbind(all_results, results)
}

return(all_results)
}

#' Generate summary table for manuscript
#'
#' @param results Simulation results
#' @param alpha Significance level
#' @return Formatted summary table
generate_summary_table <- function(results, alpha = 0.05) {
  # Calculate power
  summary <- calculate_power(results, alpha)

  # Select key scenarios for table
  key_scenarios <- c(
    "A_PH_delta=1.5",
    "B_Crossing_gamma=0.5",

```



```

    "C_Longterm_delta=2.0_cure=0.20",
    "D_Lognormal_delta=0.5"
  )

table_data <- summary %>%
  filter(scenario %in% key_scenarios) %>%
  select(scenario, method, power, convergence_rate) %>%
  spread(method, power)

return(table_data)
}

# =====
# Section 5: Visualization Functions
# =====

#' Plot power comparison across scenarios
#'
#' @param results Simulation results
#' @param alpha Significance level
#' @return ggplot object
plot_power_comparison <- function(results, alpha = 0.05) {
  summary <- calculate_power(results, alpha)

  # Extract scenario type for grouping
  summary <- summary %>%
    mutate(
      scenario_type = case_when(
        grepl("^A_", scenario) ~ "A: Proportional Hazards",
        grepl("^B_", scenario) ~ "B: Crossing Hazards",
        grepl("^C_", scenario) ~ "C: Long-term Survivors",
        grepl("^D_", scenario) ~ "D: Log-normal",
        grepl("^E_", scenario) ~ "E: Different Variances"
      ),
      method = factor(method,
        levels = c("DRM-log", "DRM-both", "DRM-boxcox",
          "Log-rank", "Peto-Prentice", "Gehan")
      )
    )

  # Filter to main scenarios for clarity
  main_scenarios <- summary %>%
    group_by(scenario_type) %>%
    slice_max(n_valid, n = 1) %>%
    pull(scenario)

  plot_data <- summary %>% filter(scenario %in% main_scenarios)

  p <- ggplot(plot_data, aes(x = scenario_type, y = power, fill = method)) +
    geom_bar(stat = "identity", position = position_dodge(0.9)) +
    geom_hline(yintercept = alpha, linetype = "dashed", color = "red") +
    labs(
      title = "Power Comparison of DRM-Based Tests vs Standard Methods",
      x = "Scenario",

```

```

    y = "Power",
    fill = "Method"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  ) +
  scale_fill_brewer(palette = "Set1")

return(p)
}

#' Plot power vs sample size
#'
#' @param results_list List of results for different sample sizes
#' @return ggplot object
plot_power_vs_n <- function(results_list) {
  all_summary <- data.frame()

  for (n in names(results_list)) {
    summary <- calculate_power(results_list[[n]])
    summary$n_actual <- as.numeric(n)
    all_summary <- rbind(all_summary, summary)
  }

  # Filter to a specific scenario for illustration
  scenario_choice <- "A_PH_delta=1.5"
  plot_data <- all_summary %>%
    filter(scenario == scenario_choice) %>%
    mutate(method = factor(method,
                           levels = c("DRM-log", "DRM-both", "DRM-boxcox",
                                       "Log-rank", "Peto-Prentice", "Gehan")))

  p <- ggplot(plot_data, aes(x = n_actual, y = power, color = method, group = method)) +
    geom_line(size = 1.2) +
    geom_point(size = 2) +
    labs(
      title = "Power vs Sample Size (Scenario A: Proportional Hazards)",
      x = "Sample Size per Group",
      y = "Power",
      color = "Method"
    ) +
    theme_minimal() +
    scale_color_brewer(palette = "Set1")

  return(p)
}

# =====
# Section 6: Main Execution
# =====

```

```

#' Main function to run the complete simulation study
#'
#' @param n Sample size per group
#' @param n_replications Number of replications
#' @param cens_prop Censoring proportion
#' @param n_cores Number of cores for parallel processing
#' @param save_results Whether to save results to file
#' @param output_dir Directory for output files
main_simulation <- function(n = 200, n_replications = 1000,
                           cens_prop = 0.2, n_cores = 4,
                           save_results = TRUE, output_dir = "simulation_results") {

  # Create output directory
  if (save_results && !dir.exists(output_dir)) {
    dir.create(output_dir, recursive = TRUE)
  }

  cat("=====\n")
  cat("DRM-Based Log-Rank Test Power Simulation\n")
  cat("=====\n")
  cat(sprintf("Sample size per group: %d\n", n))
  cat(sprintf("Replications: %d\n", n_replications))
  cat(sprintf("Censoring proportion: %.1f\n", cens_prop))
  cat(sprintf("Number of cores: %d\n", n_cores))
  cat("=====\n\n")

  # Run full simulation
  start_time <- Sys.time()
  results <- run_full_simulation(
    n = n,
    n_replications = n_replications,
    cens_prop = cens_prop,
    n_cores = n_cores
  )
  end_time <- Sys.time()

  cat("\n=====\n")
  cat(sprintf("Simulation completed in %.2f minutes\n",
              difftime(end_time, start_time, units = "mins")))
  cat("=====\n")

  # Save raw results
  if (save_results) {
    write.csv(results, file.path(output_dir, "simulation_results.csv"), row.names = FALSE)
  }

  # Calculate and display power
  summary <- calculate_power(results)
  cat("\n\nPower Summary:\n")
  print(summary)

  if (save_results) {
    write.csv(summary, file.path(output_dir, "power_summary.csv"), row.names = FALSE)
  }
}

```

```

}

# Generate summary table for manuscript
table_data <- generate_summary_table(results)
cat("\n\nSummary Table for Manuscript:\n")
print(table_data)

if (save_results) {
  write.csv(table_data, file.path(output_dir, "manuscript_table.csv"), row.names = FALSE)
}

# Create plots
p1 <- plot_power_comparison(results)
ggsave(file.path(output_dir, "power_comparison.pdf"), p1, width = 10, height = 6)

# Return results
return(list(
  results = results,
  summary = summary,
  table_data = table_data,
  plot = p1
))
}

# =====
# Section 7: Example Usage
# =====

# Example 1: Quick test with small number of replications
if (FALSE) {
  # Test individual functions
  test_data <- generate_scenario_A(n = 50, delta_hazard = 1.5, cens_prop = 0.2)
  test_result <- drm_lrt_test(test_data$time, test_data$delta, test_data$group,
                             basis_type = "log")
  print(test_result)

  # Quick simulation with 100 replications
  results <- run_simulation(
    scenario_name = "Test_A",
    n = 100,
    n_replications = 100,
    data_gen_func = generate_scenario_A,
    data_gen_args = list(n = 100, delta_hazard = 1.5, cens_prop = 0.2),
    test_methods = c("drm_log", "logrank"),
    n_cores = 2
  )
  print(calculate_power(results))
}

# Example 2: Full simulation (may take time)
if (FALSE) {
  # Run full simulation with 500 replications (for faster testing)
  results_full <- main_simulation(

```

```

    n = 200,
    n_replications = 500,
    cens_prop = 0.2,
    n_cores = 4,
    save_results = TRUE,
    output_dir = "simulation_results"
  )
}

# Example 3: Check size under null hypothesis
if (FALSE) {
  # Generate data under null (delta_hazard = 1)
  n_null <- 200
  n_reps_null <- 500

  null_results <- run_simulation(
    scenario_name = "Null_H0",
    n = n_null,
    n_replications = n_reps_null,
    data_gen_func = generate_scenario_A,
    data_gen_args = list(n = n_null, delta_hazard = 1, cens_prop = 0.2),
    test_methods = c("drm_log", "drm_both", "drm_boxcox", "logrank"),
    n_cores = 4
  )

  # Check type I error
  null_summary <- calculate_power(null_results)
  print("Type I error rates (should be ~0.05):")
  print(null_summary %>% select(method, power))
}

# =====
# Section 8: Additional Utility Functions
# =====

#' Compute empirical size and power with confidence intervals
#'
#' @param results Simulation results
#' @param alpha Significance level
#' @param conf_level Confidence level
#' @return Summary with confidence intervals
calculate_power_ci <- function(results, alpha = 0.05, conf_level = 0.95) {
  z <- qnorm(1 - (1 - conf_level) / 2)

  summary <- results %>%
    group_by(scenario, n, method) %>%
    summarise(
      power = mean(reject, na.rm = TRUE),
      se = sqrt(power * (1 - power) / sum(!is.na(reject))),
      lower = power - z * se,
      upper = power + z * se,
      n_valid = sum(!is.na(reject)),
      .groups = "drop"
    )
}

```

```

    )

    return(summary)
}

#' Check convergence rates
#'
#' @param results Simulation results
#' @return Data frame with convergence rates by method
check_convergence <- function(results) {
  conv_summary <- results %>%
    group_by(method) %>%
    summarise(
      convergence_rate = mean(converged, na.rm = TRUE),
      n_total = n(),
      .groups = "drop"
    )
  return(conv_summary)
}

#' Save formatted results for manuscript
#'
#' @param results Simulation results
#' @param output_dir Directory to save files
save_manuscript_outputs <- function(results, output_dir = "manuscript_outputs") {
  if (!dir.exists(output_dir)) {
    dir.create(output_dir, recursive = TRUE)
  }

  # Power summary
  power_summary <- calculate_power(results)
  write.csv(power_summary, file.path(output_dir, "power_summary.csv"), row.names = FALSE)

  # Table for manuscript (Table 1)
  table_data <- results %>%
    filter(scenario %in% c("A_PH_delta=1.5", "B_Crossing_gamma=0.5",
      "C_Longterm_delta=2.0_cure=0.20", "D_Lognormal_delta=0.5")) %>%
    group_by(scenario, method) %>%
    summarise(
      power = mean(reject, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    spread(method, power)

  write.csv(table_data, file.path(output_dir, "manuscript_table_1.csv"), row.names = FALSE)

  # Convergence rates
  conv_summary <- check_convergence(results)
  write.csv(conv_summary, file.path(output_dir, "convergence_summary.csv"), row.names = FALSE)

  cat("Manuscript outputs saved to:", output_dir, "\n")
}

```

```
# =====  
# End of R Script  
# =====
```